

Development Guide

This project is now considered as a pure front-end website. All the backend and database functions are used to generate static front-end file before the website starts.

Structure of files

Lists almost all file categories that you need to know for writing projects in the project and their storage locations in the following:

Main structure

```
statistics_web_java\  
  
├─ src\ # project files  
  
│├─ main\ # java web files  
  
││├─ java\ # back-end  
  
││├─ com\itheima\reggie\controller" # auxiliary back-end files  
  
││├─ resources\ # front-end  
  
││├─ common\ # scripts, styles, dictionary, assets used globally (mainly js, css, json)  
  
│││├─ common\js # Globally used javascript (js)  
  
│││├─ common\css # Globally used css (css)  
  
│││├─ common\json # Globally used json (json)  
  
│││├─ common\asset # Globally used other asset  
  
│││├─ common\js # Globally used javascript (js)  
  
││├─ component\ # Components used globally (mainly html, json)  
  
││├─ i18n\ # translation files, json text files (json)  
  
││├─ main\ # web structure of course and concept (mainly html, css, js)  
  
│││├─ main\course # course file  
  
│││├─ main\concept # concept file  
  
││├─ subpages\ # web structure of floating subpages (mainly html, css, js)  
  
││├─ index.html # The entrance of whole website  
  
└─ py\ # auxiliary python files (mainly python)
```

Course folder structure

For sections inside course, it is ordered like the following form:

```
course\  
  
├─ Introduction\ # Folder for introduction section  
  
|├─ page\ # include all the html, this is necessary, Only capitalize the first letter of a sentence  
  
|├─ js\ # JavaScript used in this section, normally not necessary  
  
|├─ style\ # CSS used in this section, normally not necessary  
  
|├─ img\ # image used in this section, normally not necessary  
  
|├─ json\ # Json used in this section, normally not necessary  
  
|├─ asset\ # Other assets used in this section, normally not necessary  
  
├─ 1. Random Sample\ # Must start with number, Capitalize the first letter of each word  
  
|├─ ...  
  
└─ ...
```

Concept structure

For sections in sider concept, it is ordered like the following form:

```
concept\  
  
├─ Introduction\ # Folder for introduction section  
  
|├─ page\ # include all the html, this is necessary  
  
||├─ Concept introduction.html # Main page of concept  
  
|├─ js\ # JavaScript used in this section, normally not necessary  
  
|├─ style\ # CSS used in this section, normally not necessary  
  
|├─ img\ # image used in this section, normally not necessary  
  
|├─ json\ # Json used in this section, normally not necessary  
  
|├─ asset\ # Other assets used in this section, normally not necessary  
  
├─ Distribution\ # Must start with number  
  
|├─ ...
```

```
|└─ page\ # include all the html, this is necessary  
  
||└─ ...  
  
||└─ section_index.html # Main page of this section  
  
└─ ...
```

For subpages, if subpage of such concept exists, then it locates in the same relative location as it in concept folder.

i18n folder structure

```
i18n\  
  
└─ en\  
  
|└─ ..... # Repeat of structure of other files in main  
  
└─ ja\  
  
|└─ ..... # Repeat of structure of other files in main  
  
└─ .....
```

Main functional mechanism

Backend static file construction function

1. Course controller

Include functions to build component of catalog of concept and course, refresh the buttons in the introduction page of course and concept. Generate section_index page of each concept section. **When you add or delete sections, or change the name of the folder and html, please run this file again to refresh them.**

If you are adding chapter or sections at last, it would be Ok because it will not cause so many changes, but if you are changing order or inserting section folder, then you had better not to run on your own branch but ask owner to run it on main branch!

2. AddChapterDataAttr

Include function to give right chapter number to each html in course section. It will find the html and give chapter number according to the file course.json generated by course controller. Please run this after you run course controller when you change anything about the number of the chapter.

3. UpdateKeyword (not realized yet)

Update keyword notation in json file in i18n according to current keyword list and translation.

If you change a word that is used very often, it will cause many changes in everywhere of the project, so you had better not to run on your own branch but ask owner to run it on main branch!

Component, script and style references

For globally used style of a concrete file category, we do not refer them one by one, by use a js to inject them in once. For example, for all course files, they all require certain type of component, js and style, we inject them by course.js, for example:

```
import {injectCss, injectJsSequential, loadComponent} from "/common/js/util_tool.js";

// First inject all the component
await Promise.all([
  loadComponent("/component/course_nav_rendered.html", "course-nav"),
  loadComponent("/component/language_switch.html", "language-switch"),
  loadComponent("/component/pager.html", "pager")
]);

// Inject css
injectCss([
  "/common/css/global_style.css",
  "/common/css/course_style.css",
  "/common/css/code_style.css",
  "/common/css/popup_style.css",
  "/common/css/course_nav.css",
  "/common/css/algorithm_style.css",
  "/common/css/theorem_style.css",
  "/common/css/table.css",
  "/common/css/pager.css"
]);

// add listener, waiting for i18n injected to add
document.addEventListener("i18nApplied", () => {
  injectJsSequential([
    "/common/js/mathjax.js",
    "/common/js/code_highlight.js"
  ])
});

// inject javascript
injectJsSequential([
  "/common/js/language_switch.js",
  "/common/js/keyword_popup.js",
  "/common/js/pager.js",
  "/common/js/course_nav_event_controller.js",
]);
```

If you have any style file, component or js want to be added, please add in the list. Also, please notice the inject occasion, avoiding bugs that may be brought by asynchronous loading.

i18n json load mechanism

The JavaScript: language_switch.js will automatically find the needed json file to do translation of current html. First, it will select certain language folder according to the language chosen now. For example, if you choose English:'en', it will search in the field of 'i18n\en'.

The JavaScript will load global json file and corresponding local json file, the local json file should be placed in exactly the same path in the language file:

```
async function loadAllDicts(lang) {
  const base = `/i18n/${lang}`;
  const pp = await makePageI18nUrl(lang);

  // Get current path
  const pathname = window.location.pathname || "";

  // Global json file
  const urls = [
    `${base}/keyword.json`,
    `${base}/component.json`,
  ];

  // conditional global json file
  if (pathname.startsWith("/main/course")) {
    urls.push(`${base}/catalog.json`);
  } else if (pathname.startsWith("/main/concept")) {
    urls.push(`${base}/concept_dic.json`);
  }

  // local json file
  urls.push(pp);

  const results = await Promise.allSettled(urls.map(fetchJson));
  const dicts = results.map(r => (r.status === "fulfilled" ? r.value : {}));

  // merge all the dictionary
  return Object.assign({}, ...dicts);
}
```

It is worth noticing that if you have duplicated keys, they will be merged, for example, if you have such in component.json:

```
"example.duplicate.key": "Content A"
```

And exactly the same key in local json:

```
"example.duplicate.key": "Content B"
```

Then the content in component.json will be replaced by the local json since it has higher priority now. Principally, you should avoid using any duplicate key unless you have special purpose.

Text load mechanism

The whole website use i18n to realize translation. So please do not write concrete content directly in html except it has no possibility to have any translation (Code is one of the typical example). The following is an example of **how you should write pages in this project**.

For html part, you should add like:

```
<p data-i18n="example.text1"></p>
```

Here the property 'data-i18n' assign the key to find translation in json file. So, in json file, there should be a corresponding key:

```
"example.text1":"This is an example of text in the project."
```

This will successfully insert concrete text into the html.

Sometimes, you may have properties that its content may need translation, in such case, you may add '-i18n' at the last of the name of such property. **Here is an example of writing the title of theorem environment**.

For example, we have a property 'data-title' needed to be realized in a theorem:

```
<aside class="statement theorem" data-title-i18n="example.theorem.title">
  //.....
</aside>
```

With corresponding key in json:

```
"example.theorem.title":"Example Theorem"
```

HTML template

Website page HTML template

We use the following structure as the following template :

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" /> // Encoding Mode
  <meta name="viewport" content="width=device-width, initial-scale=1" /> //Not necessary
  <title data-i18n="meta.title"></title> // title of website
  <link rel="stylesheet" href="../style/local_style.css"> // load local style css here

</head>

// Body of the website
<body>
  <div id="component1"></div>
  // load all component here with correct id...
```

```

<main>
    // Main content of the website...

</main>
<script>
    // load local js files here
</script>
<script type="module" src="/common/js/template.js" defer></script> // load global
injecting js file here
</body>

```

Sections template

Standard section

For a small section in the page, we usually use this format:

```

<!-- Section 1: Example, pg 1-3 --> \\ If possible, it will be kind to write the content and
corresponding pages here.
<section id="example" class="style"> \\ Input the id and style class may be used, if there
is no used one, you can also leave it blank.
    \\ Input content here
</section>

```

Theorem environment

If you want to use theorem environment, please make sure you have referred 'theorem_style.css' in your html some where.

```

<aside class="statement theorem" data-ch="1.1" data-title-i18n="theorem.example.title">
    \\ Input content here
</aside>

```

For a theorem environment, you must assign the **class** with "**statement xxx**", the concrete style will change when you input different one such as "statement theorem", "statement example". The **property data-ch** is the chapter number of this page. The **property data-title** is the title of this statement, please use i18n if there is no special case.

Code environment

If you want to use code environment, please make sure you have referred 'code_style.css' and 'code_highlight.js' in your html some where.

```

<pre><code data-i18n="example.code" class="language-c"></code></pre>

```

This will create a code block automatically. the class should be "language-xx". Please input the language you want at last. If there is no such property, then the language will be automatically assigned as R.

Algorithm environment

if you want to use algorithm environment, please first make sure you have referred 'algorithm_style.css' in your html some where.

```

<!-- ===== Example Algorithm ===== -->
<div class="algo">
  <div class="algo__header">Algorithm 1: Example Demonstration</div>

  <div class="algo__io">
    <span class="algo__label">Input:</span> A list of numbers \(\ x_1, x_2, \dots, x_n \)

    <br>
    <span class="algo__label">Output:</span> The processed result \(\ y \)
  </div>

  <ol class="algo__steps">
    <li>Initialize variables \(\ i \rightarrow 1 \), \(\ y \rightarrow 0 \).</li>

    <li>
      <div class="loop">
        <div class="loop__header">
          <span class="loop__type">FOR</span>
          <span class="loop__cond">i = 1 to n</span>
        </div>
        <div class="loop__body">
          <ul>
            <li>Compute \(\ y \rightarrow y + x_i^2 \).</li>
            <li>Display intermediate result if necessary.</li>
          </ul>
        </div>
      </div>
    </li>

    <li>
      <div class="loop">
        <div class="loop__header">
          <span class="loop__type">WHILE</span>
          <span class="loop__cond">y < 100</span>
        </div>
        <div class="loop__body">
          <ul>
            <li>Increase \(\ y \) by a constant value.</li>
            <li>Break if overflow occurs.</li>
          </ul>
        </div>
      </div>
    </li>

    <li>Return the final value \(\ y \).</li>
  </ol>
</div>

```

Here is the list of all the classes in this style:

.algo — The main algorithm container with a blue border and padding.

.algo_header — The header or title section of the algorithm.

.algo_io — The section describing the algorithm's input and output.

.algo_label — Bold label text used for keywords like "Input" and "Output."

.algo_steps — Ordered list containing the algorithm's main steps.

.loop — A loop block inside the algorithm, highlighted with a pink border.

.loop_header — The header line of a loop, showing type and condition.

.loop_type — The keyword showing loop type (e.g., "FOR", "WHILE").

.loop_cond — The loop condition or range expression.

.loop_body — The content inside the loop, typically showing substeps.

Global json and html files

There are json files that is almost always loaded by language_switch.js and html files serve as component being used globally. Here is a brief explanation of them.

In \component, there are:

- course.json
- concept.json

They are json files generated by back-end, please do not revise them directly. They record the file structure of course and concept.

- c_pager.html
- course_nav.html
- course_nav_rendered.html
- language_switch.html
- pager.html
- section_index_template.html

c_pager.html is used in concept pages as pager turning button. pager.html is used in course pages as page turning button. course_nav.html is the template of catalog component, and it is used to generate course_nav_rendered.html as catalog used in course pages. language_switch.html is the component on the top of the page, it should be include in every page except subpage. section_index_template.html is the template used to generate section_index.html in concept part.

In \common\json, there are:

- keyword.json

keyword.json records the information of keywords, including their name, i18n key and main page, subpage location.

In \i18n\xx\, there are:

- catalog.json
- component.json

- concept_dic.json
- keyword.json

catalog.json records the translation of section titles in course, and concept_dic.json records the translation of section titles in concept. keyword.json records the translation of keywords. component.json records the text content in component files.